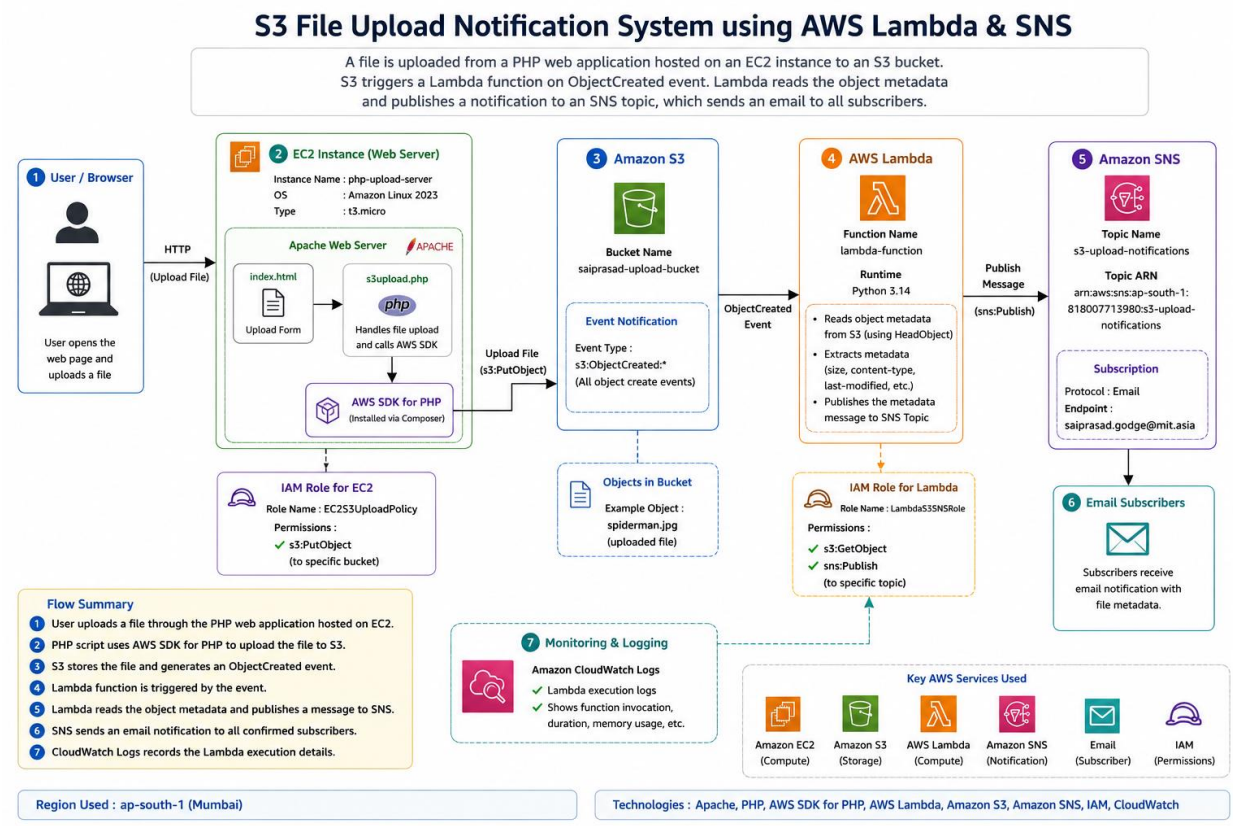
AWS Event-Driven File Upload Notification System

(using Amazon EC2, Amazon S3, AWS Lambda, Amazon SNS, and PHP)

Objective:

The objective of this project is to build an automated file upload system using AWS services. Users upload a file through a PHP web application hosted on an EC2 instance. The file is stored in an Amazon S3 bucket, which automatically triggers an AWS Lambda function. Lambda reads the file details and sends them to Amazon SNS, which then emails the subscribed users. This project demonstrates how AWS services work together to automate a real-world workflow.

System Architecture Overview:



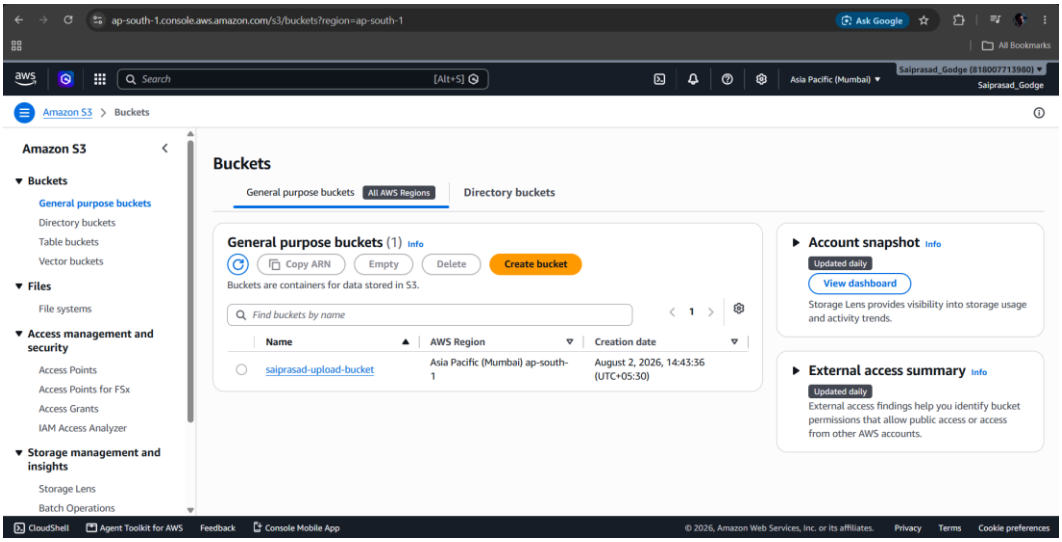
Implementation:

Step 1: Create the Amazon S3 Bucket

Create a private S3 bucket to serve as the destination repository for uploaded documents.

Action Steps:

- ✓ Navigate to S3 in the AWS Management Console and click 'Create bucket'.
- ✓ Choose a globally unique name like 'saiprasad-upload-bucket' and configure your preferred region (e.g. ap-south-1).
- ✓ Ensure 'Block all public access' is kept ENABLED. The web app uses IAM roles for secure back-end communication, eliminating the need for public access.
- ✓ Click 'Create bucket' at the bottom to finalize resource creation.

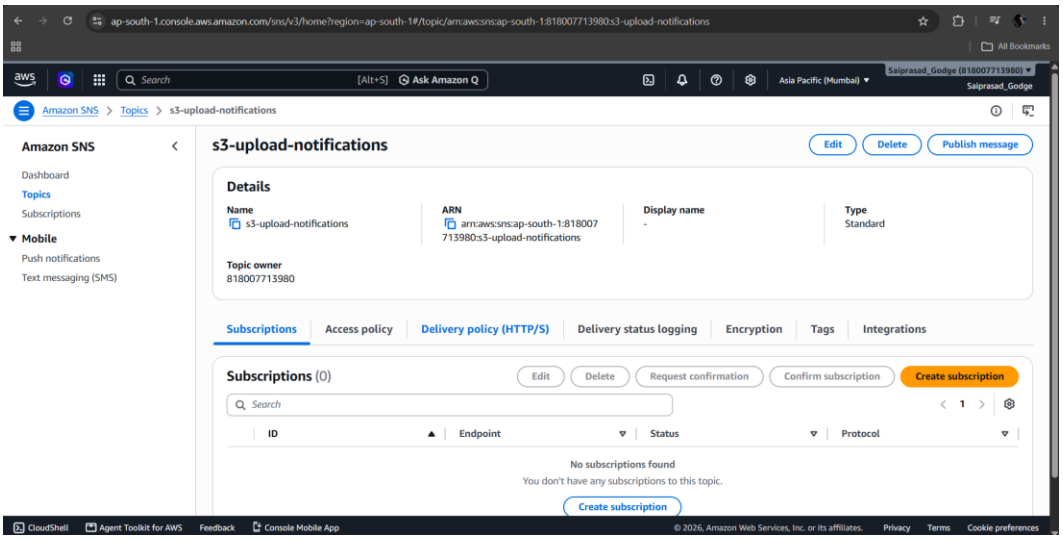


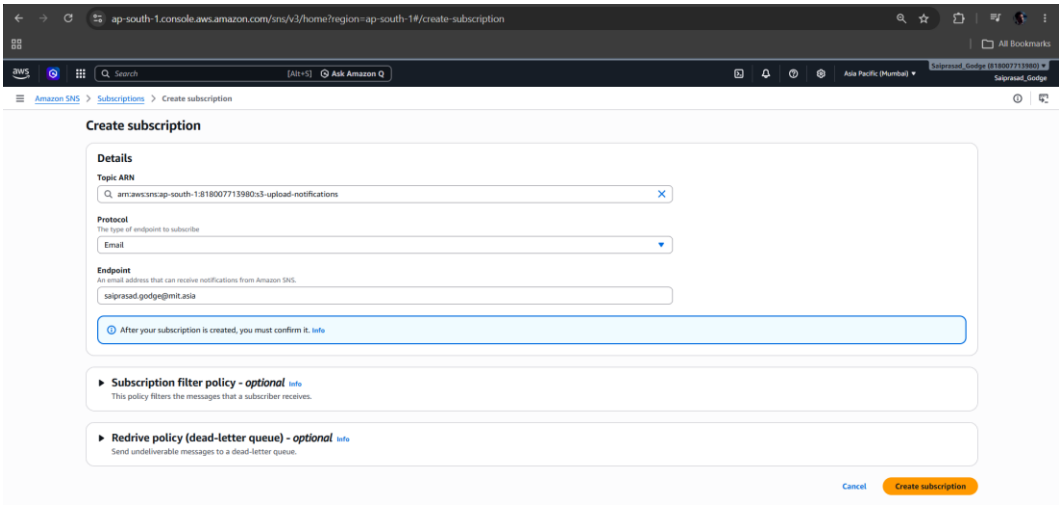
Step 2: Configure the Amazon SNS Topic and Subscriptions

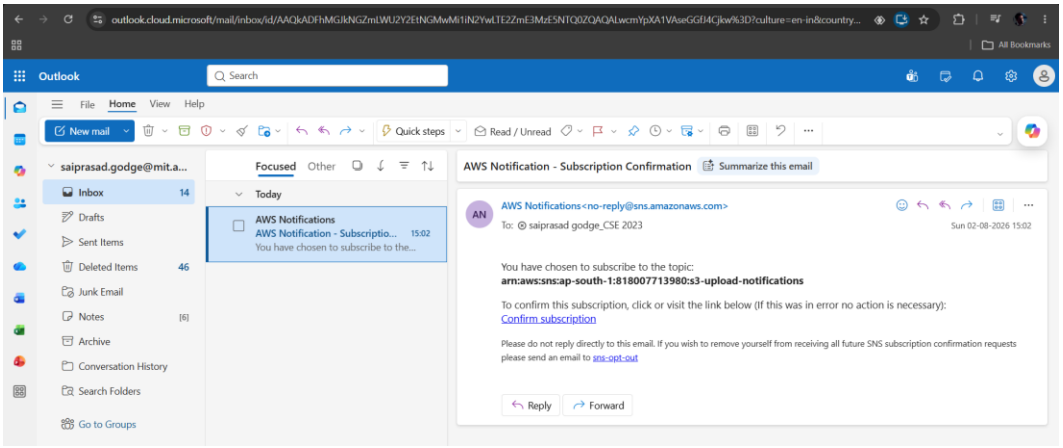
Create an SNS topic to dispatch email alerts whenever uploads occur, and set up a confirmed subscription.

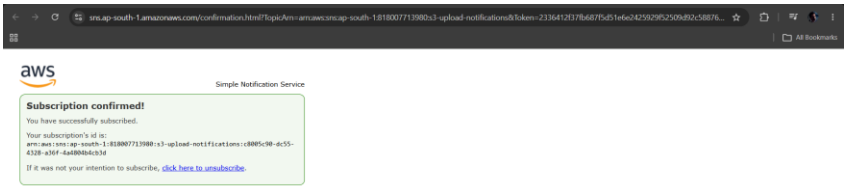
Action Steps:

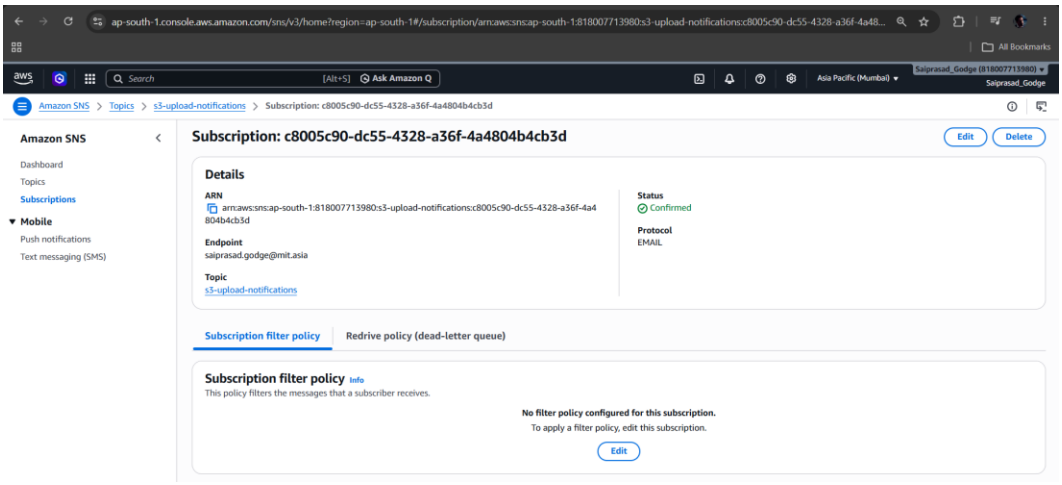
- ✓ Open the Amazon SNS Console, choose 'Topics', and click 'Create topic'.
- ✓ Select 'Standard' type and enter the topic name 's3-upload-notifications'. Leave defaults and click 'Create topic'.
- ✓ Copy the Topic ARN (e.g., arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications) as it will be required in later configurations.
- ✓ Under the topic details, click 'Create subscription', set the Protocol to 'Email', enter the subscriber's email address, and click 'Create subscription'.
- ✓ Open the subscriber's email inbox, locate the 'AWS Notification - Subscription Confirmation' email, and click 'Confirm subscription'.
- ✓ Verify in the SNS Console that the subscription status has transitioned to 'Confirmed'.


ap-south-1.console.aws.amazon.com/sns/v3/home?region=ap-south-1#/topic/arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications
Amazon SNS > Topics > s3-upload-notifications
Amazon SNS
Dashboard
Topics
Subscriptions
Mobile
Push notifications
Text messaging (SMS)
s3-upload-notifications
Edit
Delete
Publish message
Details
Name
s3-upload-notifications
ARN
arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications
Display name
-
Type
Standard
Topic owner
818007713980
Subscriptions
Access policy
Delivery policy (HTTP/S)
Delivery status logging
Encryption
Tags
Integrations
Subscriptions (0)
Edit
Delete
Request confirmation
Confirm subscription
Create subscription
Search
ID
Endpoint
Status
Protocol
No subscriptions found
You don't have any subscriptions to this topic.
Create subscription


ap-south-1.console.aws.amazon.com/sns/v3/home?region=ap-south-1#/create-subscription
Amazon SNS > Subscriptions > Create subscription
Create subscription
Details
Topic ARN
arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications
Protocol
The type of endpoint to subscribe
Email
Endpoint
An email address that can receive notifications from Amazon SNS.
saiprasad.godge@mit.asia
After your subscription is created, you must confirm it. Info
Subscription filter policy - optional Info
This policy filters the messages that a subscriber receives.
Redrive policy (dead-letter queue) - optional Info
Send undeliverable messages to a dead-letter queue.
Cancel
Create subscription


Outlook
Search
File
Home
View
Help
New mail
Quick steps
Read / Unread
saiprasad.godge@mit.a...
Inbox 14
Drafts
Sent Items
Deleted Items 46
Junk Email
Notes [6]
Archive
Conversation History
Search Folders
Go to Groups
Focused
Other
Today
AWS Notifications
AWS Notification - Subscriptio... 15:02
You have chosen to subscribe to the...
AWS Notification - Subscription Confirmation
Summarize this email
AWS Notifications<no-reply@sns.amazonaws.com>
To: saiprasad godge_CSE 2023
Sun 02-08-2026 15:02
You have chosen to subscribe to the topic:
arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications
To confirm this subscription, click or visit the link below (If this was in error no action is necessary):
Confirm subscription
Please do not reply directly to this email. If you wish to remove yourself from receiving all future SNS subscription confirmation requests please send an email to sns-opt-out
Reply
Forward


Simple Notification Service
Subscription confirmed!
You have successfully subscribed.
Your subscription's id is:
arn:aws:sns:ap-south-1:818007713980:s3-upload-notifications:c8005c90-dc55-4328-a16f-4a8804b4cb3d
If it was not your intention to subscribe, click here to unsubscribe.



Step 3: Create the AWS Lambda Function (Python 3.12)

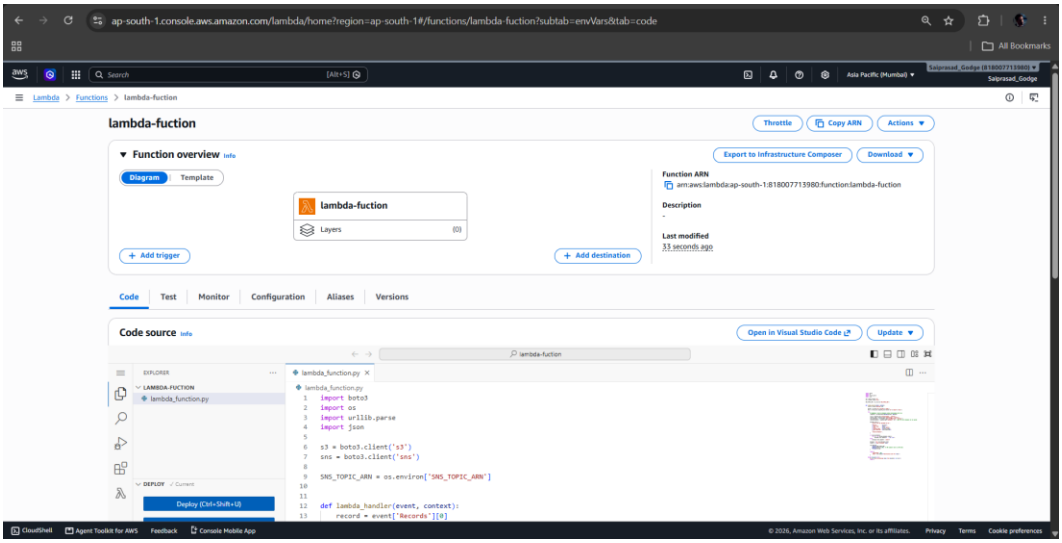
Create a serverless function that automatically runs when triggered by S3, gathers file metadata, and publishes a structured notice to SNS.

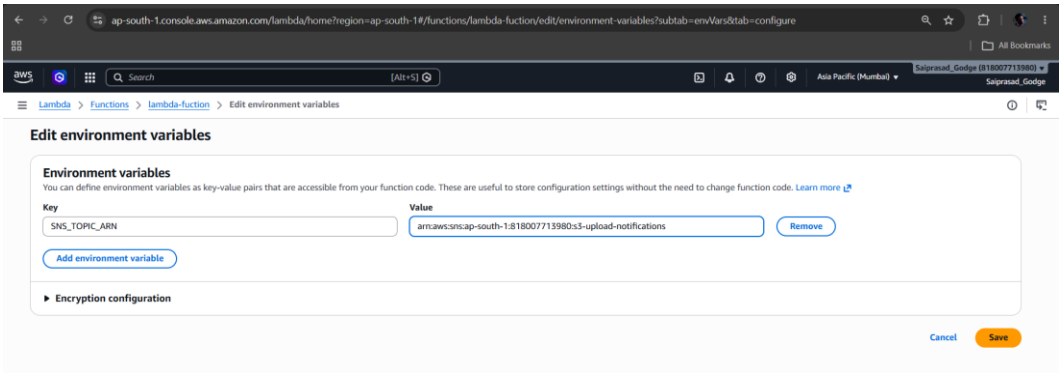
Action Steps:

- ✓ Navigate to AWS Lambda and click 'Create function'. Select 'Author from scratch'.
- ✓ Set the Function Name to 'lambda-function', choose the Runtime 'Python 3.12', and leave architecture as 'x86_64'. Click 'Create function'.
- ✓ Navigate to the code editor tab, delete the placeholder template, and paste the Python handler code.
- ✓ Click 'Deploy' to publish your function changes.

Lambda Function Source Code:

GitHub Repository: https://github.com/SAI-GODGE/aws-s3-lambda-sns-file-upload





Step 4: Configure Lambda Permissions and Environment Variables

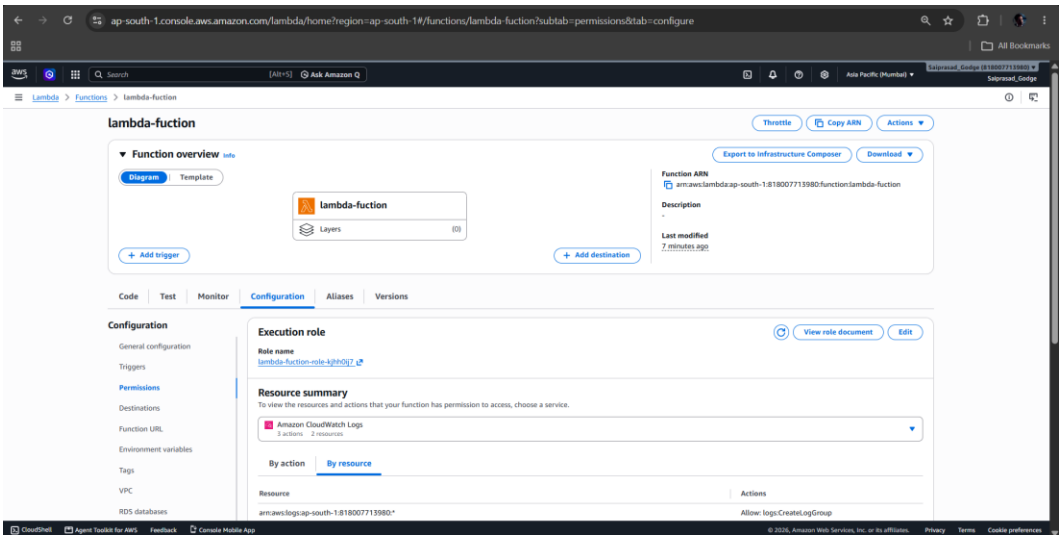
Grant Lambda permissions to perform s3:GetObject on the bucket and sns:Publish on the topic, and set its environment variables.

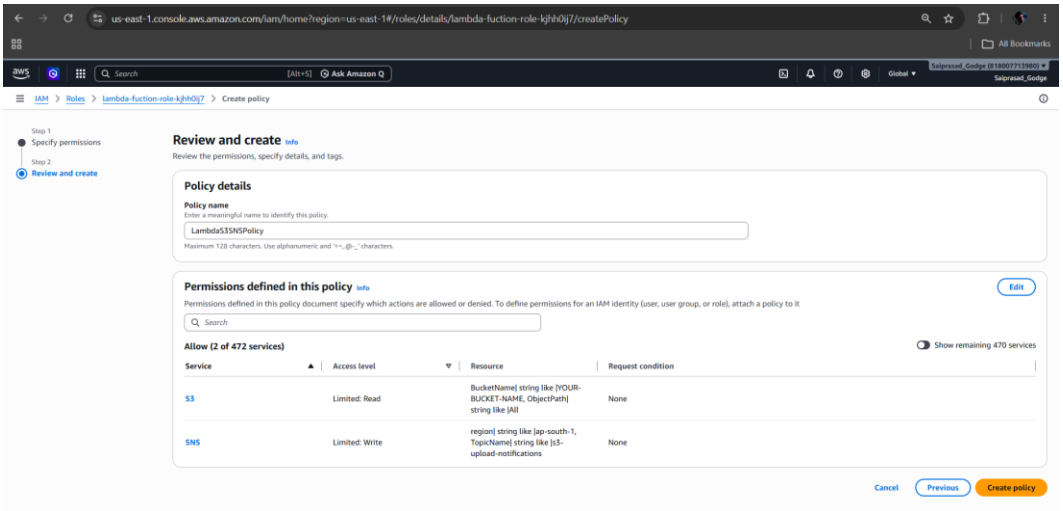
Action Steps:

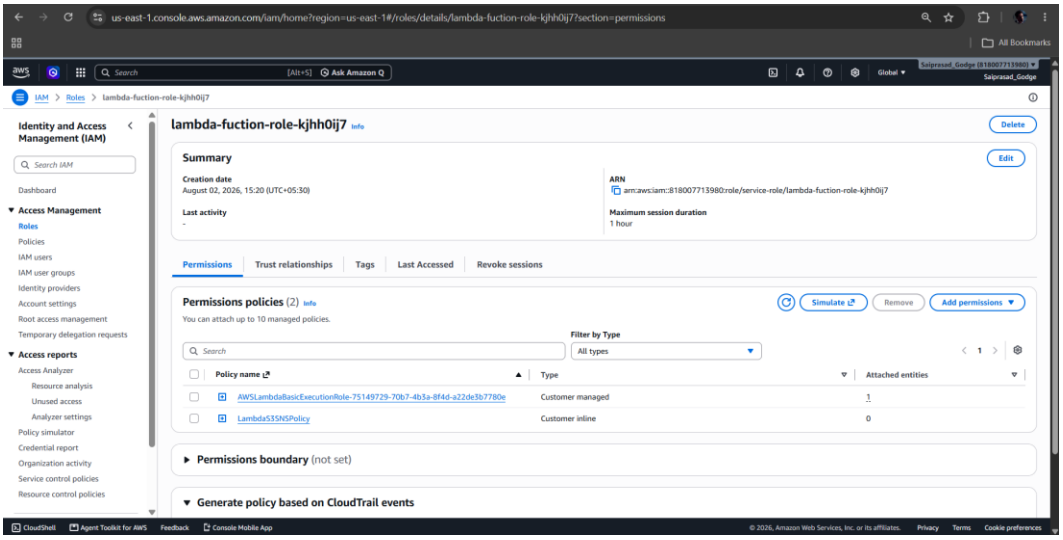
- ✓ Go to your Lambda function's 'Configuration' tab, select 'Permissions', and click the link under 'Execution role' to open the IAM role.
- ✓ In IAM, click 'Add permissions' **---->** 'Create inline policy'. Select the 'JSON' editor.
- ✓ Paste the IAM policy granting read access to the S3 bucket objects and publish rights to the SNS topic.
- ✓ Name the policy 'LambdaS3SNSPolicy' and attach it.
- ✓ Back in the Lambda console, navigate to 'Configuration' **---->** 'Environment variables' and add a key 'SNS_TOPIC_ARN' containing your SNS ARN.

Command Scripts:

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject"
      ],
      "Resource": "arn:aws:s3:::saiprasad-upload-bucket/*"
    },
    {
      "Effect": "Allow",
      "Action": [
        "sns:Publish"
      ],
      "Resource": "arn:aws:sns:ap-south-1:818007713980:s3-upload-
notifications"
    }
  ]
}
```


ap-south-1.console.aws.amazon.com/lambda/home?region=ap-south-1#/functions/lambda-fuction?subtab=permissions&tab=configure
lambda-fuction
Throttle
Copy ARN
Actions
Function overview
Export to Infrastructure Composer
Download
Diagram
Template
lambda-fuction
Layers
(0)
Add trigger
Add destination
Function ARN
arn:aws:lambda:ap-south-1:818007713980:function:lambda-fuction
Description
Last modified
7 minutes ago
Code
Test
Monitor
Configuration
Aliases
Versions
Configuration
General configuration
Triggers
Permissions
Destinations
Function URL
Environment variables
Tags
VPC
RDS databases
Execution role
View role document
Edit
Role name
lambda-fuction-role-kjhh0ij7
Resource summary
To view the resources and actions that your function has permission to access, choose a service.
Amazon CloudWatch Logs
3 actions 2 resources
By action
By resource
Resource
Actions
arn:aws:logs:ap-south-1:818007713980:*
Allow: logs:CreateLogGroup


us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/details/lambda-fuction-role-kjhh0ij7/createPolicy
IAM > Roles > lambda-fuction-role-kjhh0ij7 > Create policy
Step 1
Specify permissions
Step 2
Review and create
Review and create
Review the permissions, specify details, and tags.
Policy details
Policy name
Enter a meaningful name to identify this policy.
LambdaS3SNSPolicy
Maximum 128 characters. Use alphanumeric and '+=,.@-_' characters.
Permissions defined in this policy
Permissions defined in this policy document specify which actions are allowed or denied. To define permissions for an IAM identity (user, user group, or role), attach a policy to it
Edit
Search
Allow (2 of 472 services)
Show remaining 470 services
Service
Access level
Resource
Request condition
S3
Limited: Read
BucketName| string like |YOUR-BUCKET-NAME, ObjectPath| string like |All
None
SNS
Limited: Write
region| string like |ap-south-1, TopicName| string like |s3-upload-notifications
None
Cancel
Previous
Create policy

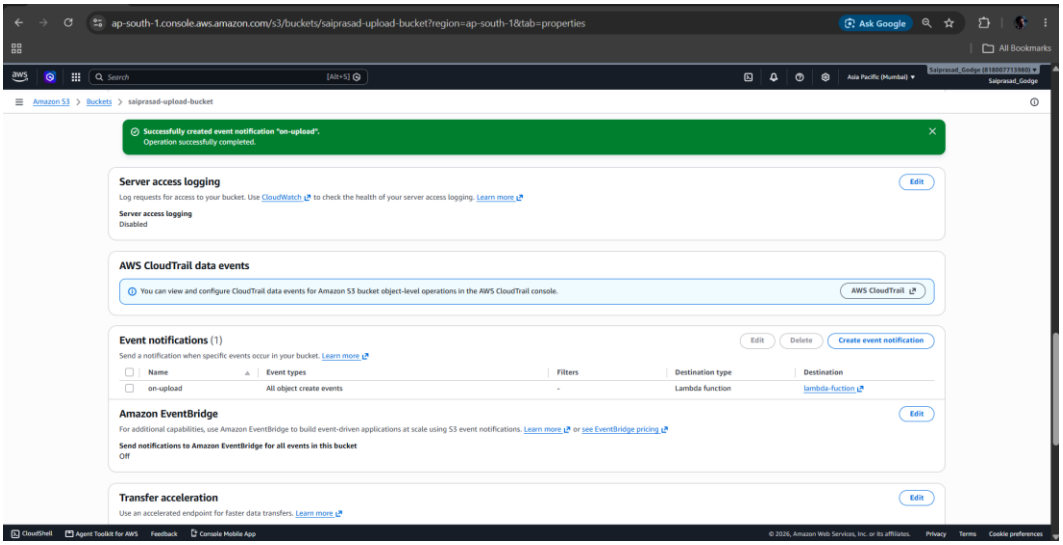

us-east-1.console.aws.amazon.com/iam/home?region=us-east-1#/roles/details/lambda-fuction-role-kjhh0ij7?section=permissions
IAM > Roles > lambda-fuction-role-kjhh0ij7
Identity and Access Management (IAM)
Search IAM
Dashboard
Access Management
Roles
Policies
IAM users
IAM user groups
Identity providers
Account settings
Root access management
Temporary delegation requests
Access reports
Access Analyzer
Resource analysis
Unused access
Analyzer settings
Policy simulator
Credential report
Organization activity
Service control policies
Resource control policies
lambda-fuction-role-kjhh0ij7
Delete
Summary
Edit
Creation date
August 02, 2026, 15:20 (UTC+05:30)
ARN
arn:aws:iam::818007713980:role/service-role/lambda-fuction-role-kjhh0ij7
Last activity
-
Maximum session duration
1 hour
Permissions
Trust relationships
Tags
Last Accessed
Revoke sessions
Permissions policies (2)
You can attach up to 10 managed policies.
Simulate
Remove
Add permissions
Filter by Type
Search
All types
Policy name
Type
Attached entities
AWSLambdaBasicExecutionRole-75149729-70b7-4b3a-8f4d-a22de3b7780e
Customer managed
1
LambdaS3SNSPolicy
Customer inline
0
Permissions boundary (not set)
Generate policy based on CloudTrail events

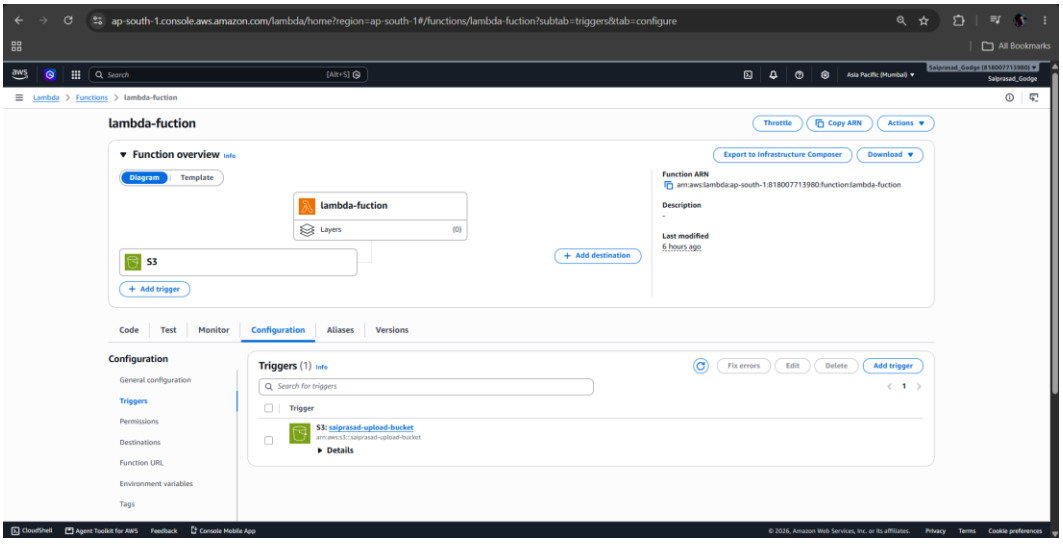
Step 5: Set Up S3 Bucket Event Notification Trigger

Connect Amazon S3 to AWS Lambda so that every file upload automatically triggers the Lambda function.

Action Steps:

- ✓ Go to your S3 bucket page, navigate to the 'Properties' tab, and scroll to 'Event notifications'. Click 'Create event notification'.
- ✓ Fill in the settings: Event name 'on-upload', verify prefix/suffix are empty (to capture all files), select Event type 'All object create events (s3:ObjectCreated:*)'.
- ✓ Set Destination to 'Lambda function' and select your 'lambda-function' from the list. Save changes.
- ✓ Open your Lambda function and verify that the S3 trigger block is successfully displayed in the Function Overview.





Step 6: Launch the Amazon EC2 Web Server

Deploy an Amazon Linux 2023 virtual machine and configure an Apache web server with PHP support.

Action Steps:

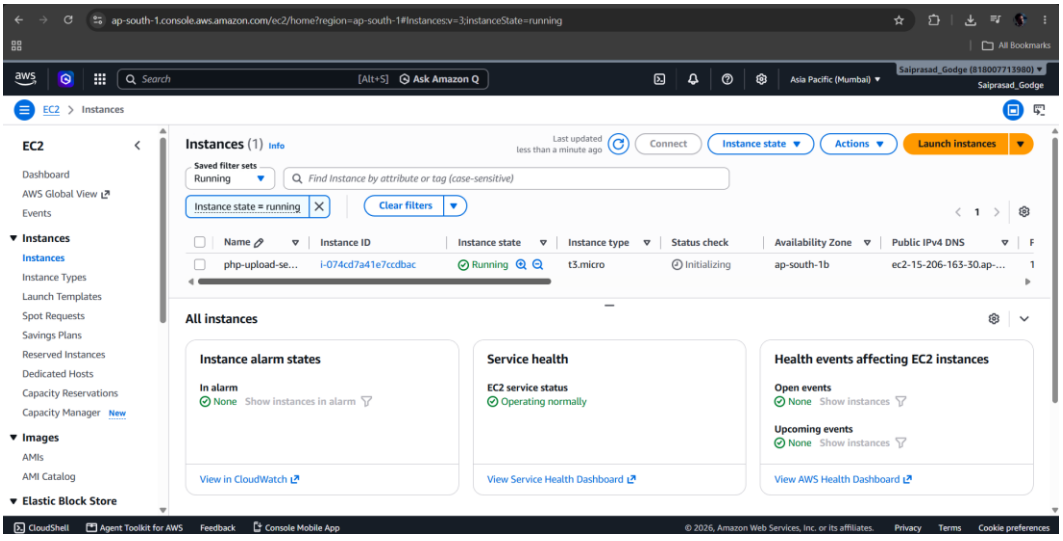
- ✓ Open the EC2 Dashboard and click 'Launch Instance'. Name it 'php-upload-server'.
- ✓ Select 'Amazon Linux 2023' (Free Tier eligible t3.micro) and associate your key pair (e.g. 'php-s3-lambda-key').
- ✓ Create a security group allowing inbound HTTP (port 80) from anywhere and SSH (port 22) from your IP.
- ✓ Launch the instance, copy its public IP, and log in via SSH.
- ✓ Update packages and install Apache, PHP, and unzip command line dependencies: 'sudo dnf update -y' and 'sudo dnf install -y php php-cli httpd unzip'.
- ✓ Enable and boot the Apache service: 'sudo systemctl enable --now httpd'.
- ✓ In a web browser, load 'http://<EC2-Public-IP>' to verify that the default Apache page is displayed.

Command Scripts:

```
# SSH into the instance
ssh -i php-s3-lambda-key.pem ec2-user@<EC2-Public-IP>

# Install Apache and PHP
sudo dnf update -y
sudo dnf install -y php php-cli httpd unzip

# Start Apache
sudo systemctl enable --now httpd
```





It works!

Step 7: Install Composer and AWS SDK for PHP

Install Composer and the AWS SDK for PHP in the web application directory to enable the PHP application to interact with AWS services.

Action Steps:

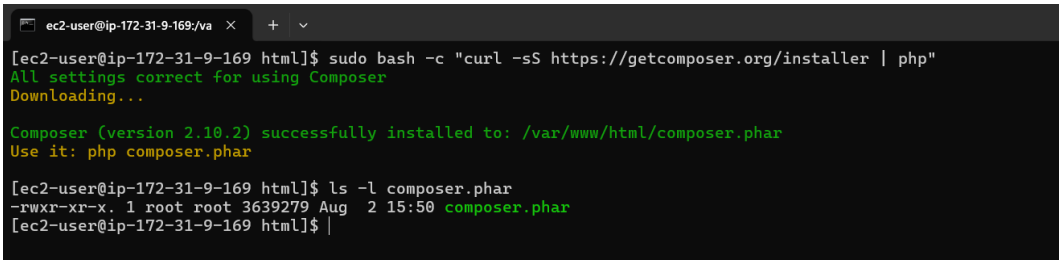
- ✓ Navigate to the web root: 'cd /var/www/html'.
- ✓ Install Composer locally. Since the web root is owned by root, run: 'sudo bash -c "curl -sS https://getcomposer.org/installer | php"'.
- ✓ Use Composer to pull down the official AWS SDK packages: 'sudo php composer.phar require aws/aws-sdk-php'.

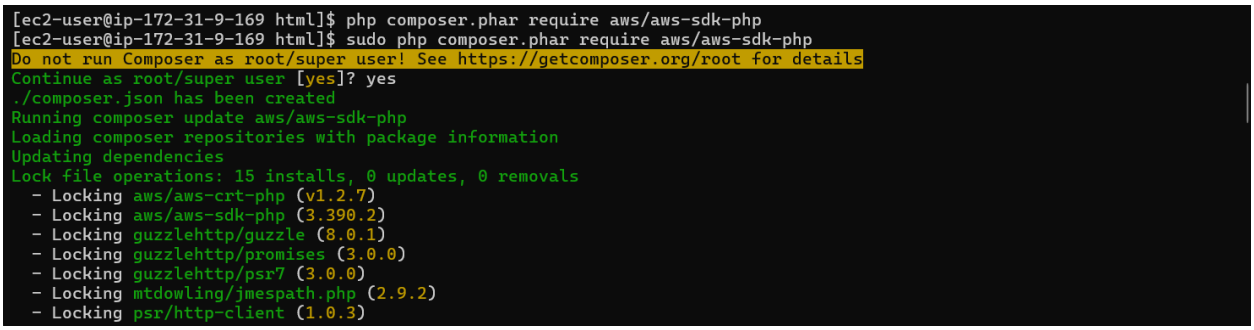
Command Scripts:

```
cd /var/www/html

# Download Composer
sudo bash -c "curl -sS https://getcomposer.org/installer | php"

# Download AWS SDK for PHP
sudo php composer.phar require aws/aws-sdk-php
```



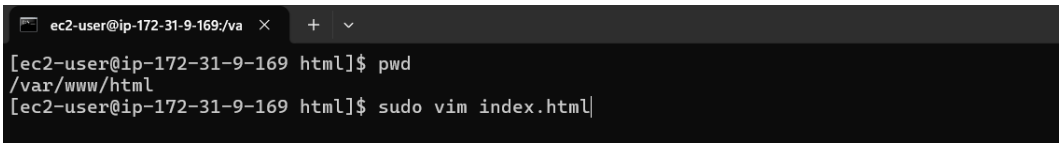


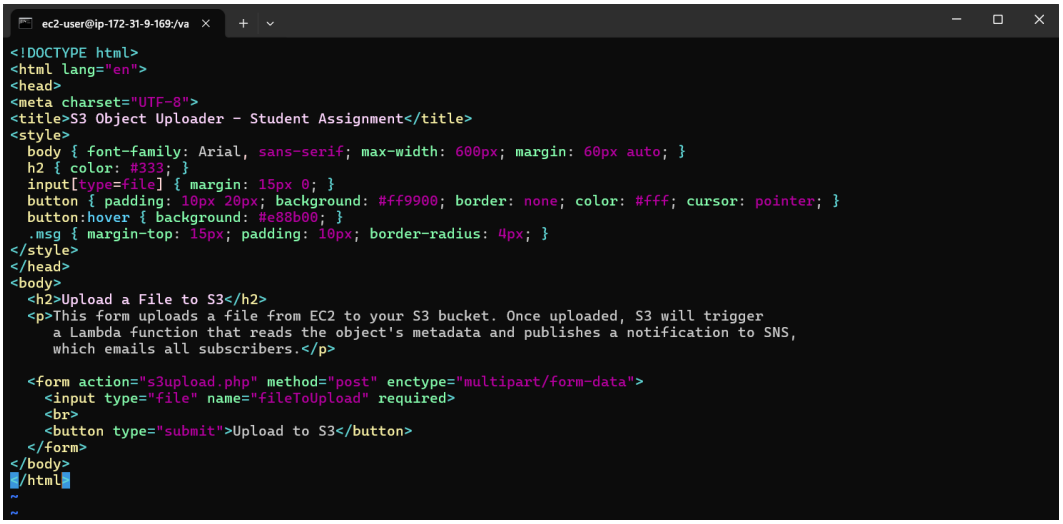
Step 8: Develop the PHP Web Application

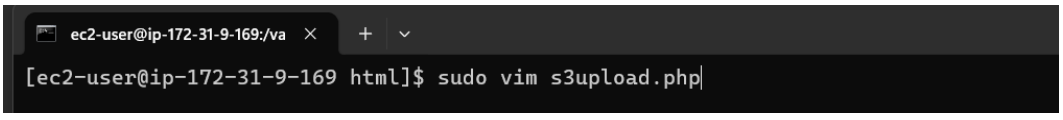
Create the PHP web application that allows users to upload files to the Amazon S3 bucket.

Action Steps:

- ✓ In the **/var/www/html** directory, create an **index.html** file containing a file upload form that submits the selected file to **s3upload.php**.
- ✓ Create a **s3upload.php** file and include the AWS SDK autoloader using: require 'vendor/autoload.php';
- ✓ Configure the **Amazon S3 bucket name** and **AWS Region** in the s3upload.php file.
- ✓ Create an **Amazon S3 Client** using the AWS SDK. The application automatically uses the IAM role attached to the EC2 instance for authentication.
- ✓ Use the **putObject()** method to upload the selected file to the Amazon S3 bucket along with the required metadata.







S3upload.php file Source Code:

GitHub Repository: https://github.com/SAI-GODGE/aws-s3-lambda-sns-file-upload

Step 9: Configure and Attach an IAM Role to the EC2 Instance

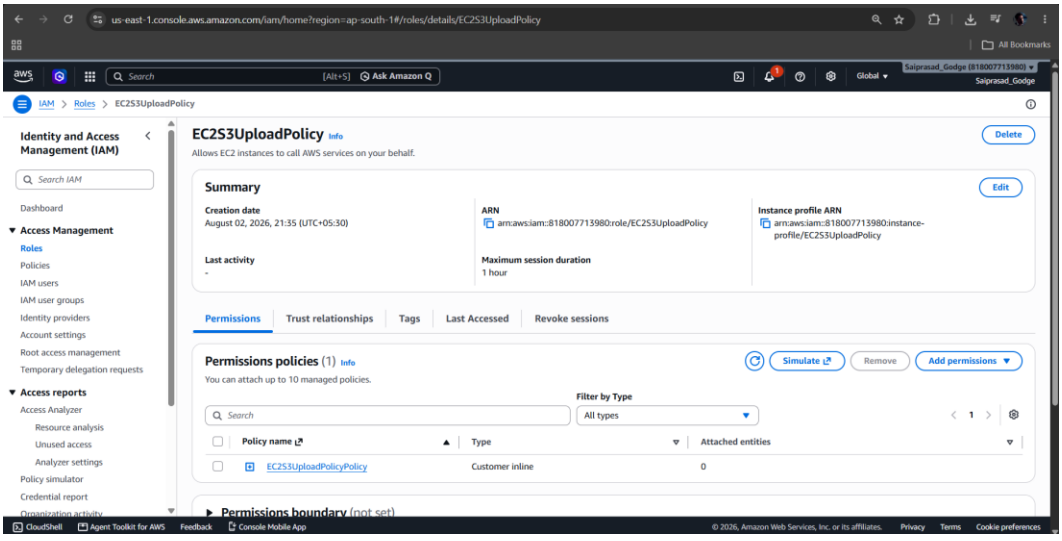
Create and attach an IAM role that allows the EC2 instance to upload files to the Amazon S3 bucket without storing AWS credentials in the application.

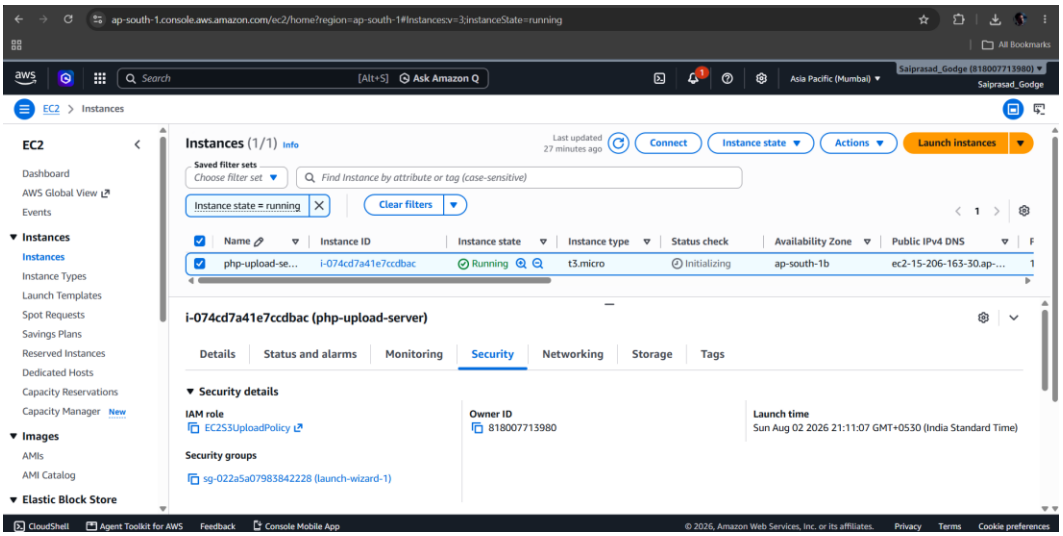
Action Steps:

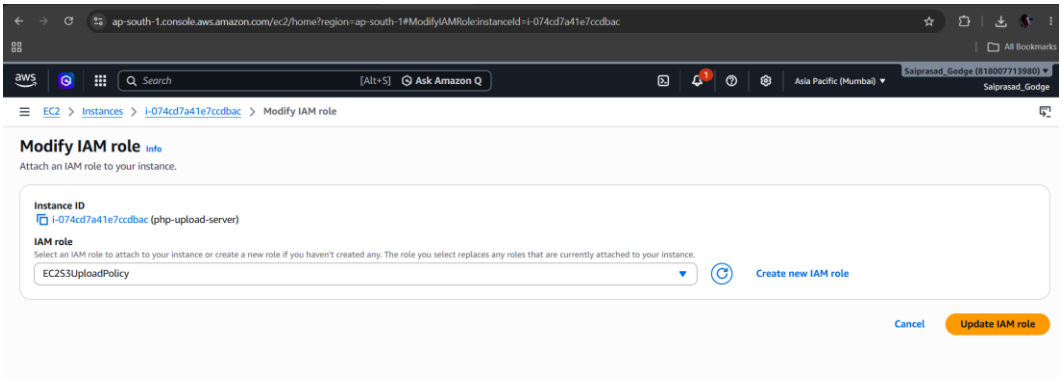
- ✓ Navigate to IAM ----> Roles ----> 'Create role'. Under Trusted Entity, choose 'AWS Service' and select 'EC2' use case.
- ✓ Click 'Create policy', switch to the 'JSON' editor, and insert a statement granting 's3:PutObject' for your specific bucket.
- ✓ Name the policy 'EC2S3UploadPolicy', save it, and select it to attach to the role 'EC2S3UploadRole'.
- ✓ Go to the EC2 console, select your 'php-upload-server' instance, and choose 'Actions'----> 'Security' ----> 'Modify IAM role'.
- ✓ Choose 'EC2S3UploadRole' and click 'Update IAM role'.
- ✓ Return to your EC2 console session and run: 'aws sts get-caller-identity' to verify the instance assumes the IAM role.

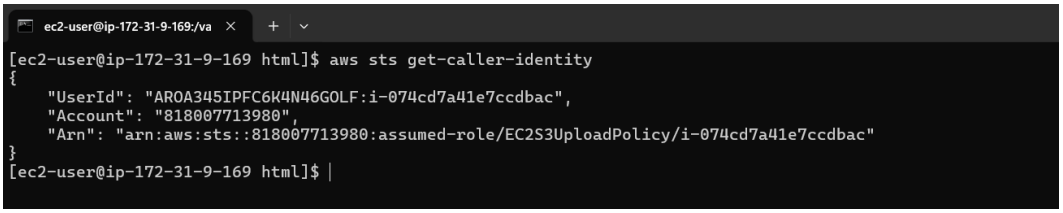
Command Scripts:

```
// IAM Policy JSON (EC2S3UploadPolicy)
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:PutObject"
      ],
      "Resource": "arn:aws:s3:::saiprasad-upload-bucket/*"
    }
  ]
}
```









Step 10: Test and Verify the Application

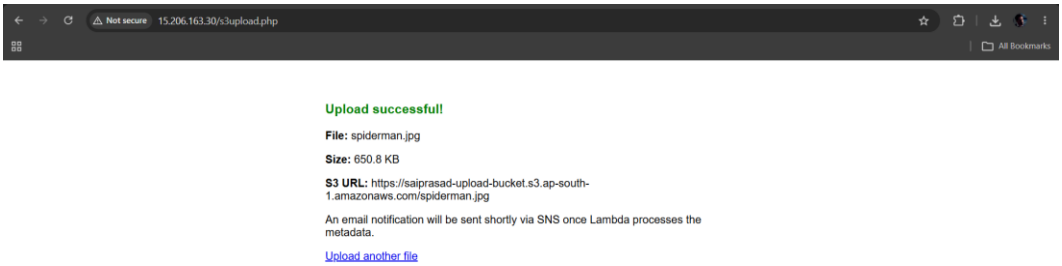
Execute a full end-to-end test and inspect the event notifications, CloudWatch logs, S3 bucket storage, and Outlook inbox notifications.

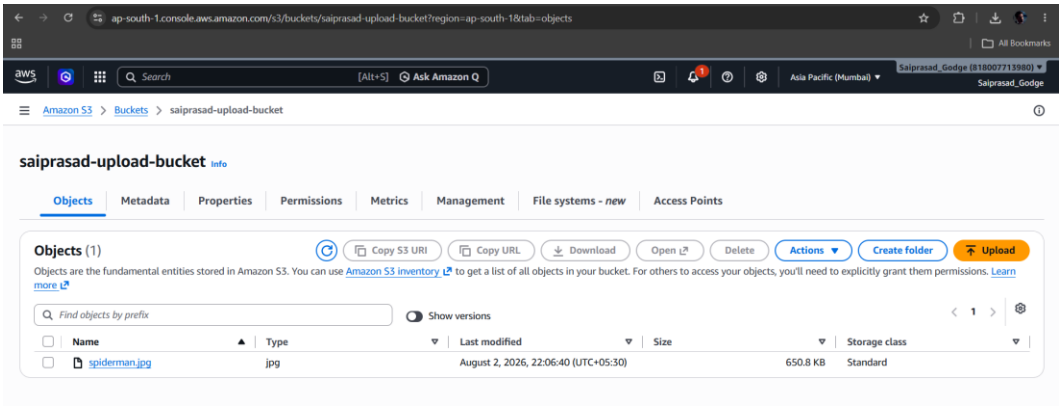
Action Steps:

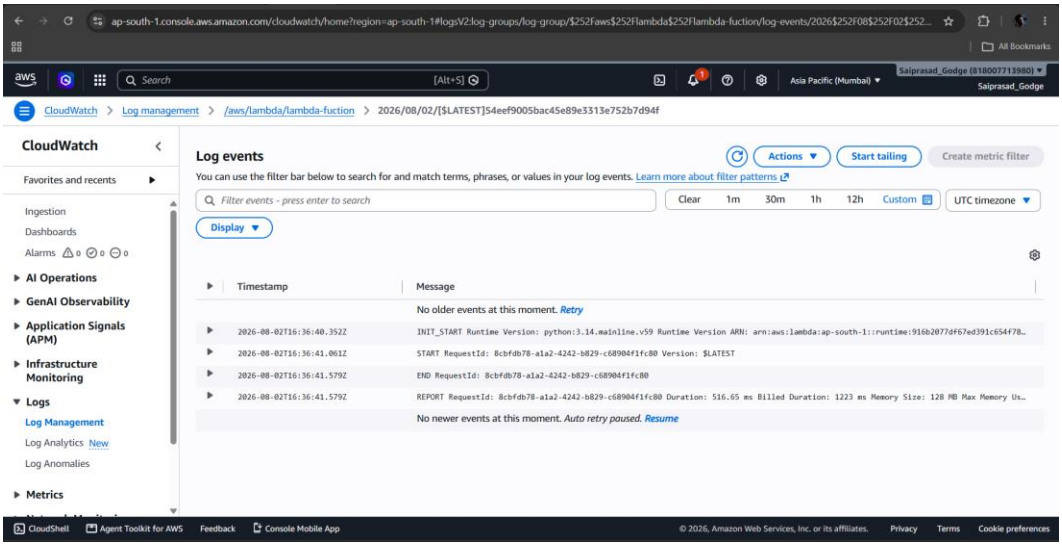
- ✓ Navigate to 'http://<EC2-Public-IP>/index.html' in your browser.
- ✓ Choose a test file, e.g. 'spiderman.jpg', and click 'Upload to S3'.
- ✓ Verify that the page redirects to a success screen detailing the uploaded file name, size, and the direct S3 URL.
- ✓ Log into the AWS Console and check your S3 bucket objects to verify that the file exists.
- ✓ Go to CloudWatch Logs under the Lambda log group '/aws/lambda/lambda-function' to trace invocation logs showing execution stats.

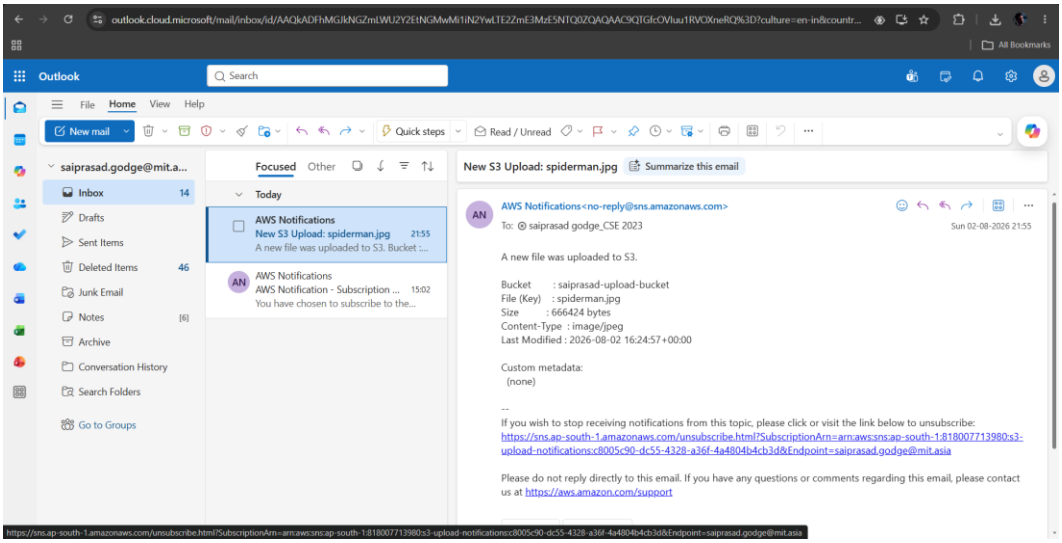
✓ Open your Outlook email client to verify you received a notification containing key metadata fields from the S3 file upload.











Result:

The AWS event-driven file upload notification system was implemented successfully. Files uploaded through the PHP web application on the EC2 instance were stored securely in the Amazon S3 bucket. Each file upload automatically triggered the AWS Lambda function, which retrieved the file metadata and published it to an Amazon SNS topic. The SNS service then sent an email notification containing the uploaded file details. The successful execution of the Lambda function was verified through Amazon CloudWatch logs, confirming that all AWS services worked together as expected.